

论文模板 02 · 云原生与 Serverless (FaaS) 架构

配套素材见 [00-项目素材库.md](#)。本主题近三年出现频率极高，强烈建议准备。

一、真题常见问法

变体 A（云原生整体）

请论述你参与的某系统的云原生架构设计，包括容器化、微服务、持续交付、DevOps 等方面的实践，并说明其带来的价值。

变体 B（Serverless 专项）

请论述 Serverless（无服务器）架构的适用场景与设计要点，结合你参与的项目说明如何划分函数粒度、如何处理冷启动与状态管理。

变体 C（弹性与成本）

请论述你如何利用云平台能力实现系统的弹性伸缩与成本优化。

判断：题目提「容器/K8s/CI-CD」→ 走变体 A 路线；提「函数计算/事件驱动/按量付费」→ 走变体 B 路线（本项目 Lambda 素材极强）。

二、摘要模板（约 300 字）

本文论述了本人参与的 H.I.S. BASE HOME 旅游业务平台的云原生架构设计与实现。

该平台部署于 AWS 东京区域，承载预约管理、客户消息、通知发送等业务。

本人担任架构师，负责容器化方案设计、Serverless 数据同步链路设计与交付流水线建设。

针对【数据同步与主链路解耦】、【函数粒度划分】、【多云资源治理】三大挑战，

本文从【事件驱动解耦】、【按业务域拆分函数】、【基础设施即代码】三个维度进行设计：

以 DynamoDB Stream 与 SQS 作为事件源，驱动 14 个按业务域划分的 Lambda 函数完成数据同步，使主链路仅写 DynamoDB 即可返回，达成分钟级最终一致性；

每个函数独立镜像、独立流水线、独立部署，实现故障域隔离。

系统上线后【 填效果】，验证了方案的可行性。

三、正文五段框架

第 1 段 · 项目背景与为何选择云原生（约 400 字）

关键：不要一上来就说「我们上云了」，要写驱动力。

旅游业务的典型特征是**强季节性**：长假前预约量激增，平峰期骤降。传统固定容量的部署方式在旺季需要按峰值配置资源，淡季则大量闲置，资源利用率长期偏低。这一矛盾是推动我们采用云原生架构的直接动因。

本项目可直接用：

BASE HOME 部署于 AWS ap-northeast-1，采用 Java 21 + Spring Boot 3.5.7，各模块以容器镜像方式交付（基础镜像统一为内部 Java 21 镜像），数据同步链路由 14 个 Python 3.13 Lambda 函数承担，配置统一存放于 SSM Parameter Store，避免密钥硬编码。

第 2 段 · 架构设计 (约 800 字)

分三层论述：

第一层：容器化与交付

Web (8080)、Service (gRPC 6565)、Batch (无 Web 容器，`spring.main.web-application-type: none`) 各自独立 Dockerfile 与镜像，镜像基于集团统一基础镜像，保证运行时基线一致。本地研发环境通过 TestContainers 拉起 MySQL，通过 LocalStack (:4566) 模拟 DynamoDB 与 S3，使「本地能跑通」与「云上能跑通」的技术栈一致，消除了环境差异导致的缺陷。

第二层：Serverless 数据同步 (本文核心)

系统的写入主链路落在 DynamoDB，而报表与既有系统仍需从 MySQL 查询。若在 Java 服务中同步写两侧，主链路响应时间会被拉长，且同步逻辑的扩容需求会绑架整个服务的资源配置。因此我把同步链路设计为**事件驱动的 Serverless 架构**：

- **事件源**：DynamoDB Stream (写后事件) + SQS (业务消息队列)
- **计算单元**：14 个 Lambda 函数，按业务域拆分
- **配置源**：SSM Parameter Store
- **可观测**：`aws_lambda_powertools` 提供结构化日志、Tracer 链路追踪与 SQS/Stream 批量处理工具

按触发方式可分为两类：| 触发方式 | 函数 | |---|---| | DynamoDB Stream |

`import_document_information`、`import_message_thread`、`import_parental_consent`、
`import_travel_information`、`send_travel_information` || SQS | `import_booking_information`、
`import_mail_information`、`import_notification`、`import_wallet_notification`、
`import_wallet_status`、`update_travel_status` || 定时 | `send_email_footer`、
`send_email_template`、`send_todo_information` |

权衡说明：该设计以**最终一致性**替代强一致性——主链路返回时，MySQL 侧数据尚未更新。评估后我们认为对报表与通知场景可接受，而预约写入等强一致场景仍走关系库事务，不做异步化。

第三层：函数粒度划分原则

我没有按「一张表一个函数」划分，而是按**业务域**划分。理由：同一业务域的实体更新往往需要一起处理（如旅行信息变更需同步 申请人与预约数据），按表拆会导致函数间编排复杂、且同一业务逻辑被多次触发。按域拆分使每个函数内部可完成完整的业务处理，同时故障域按业务隔离——某个域的函数异常不会波及其他业务。

第 3 段 • 关键技术实现 (约 700 字)

要素一：事件驱动的解耦

主链路只写 DynamoDB 即返回，同步由 Stream 异步接管，把「写数据库」与「同步数据」两个关注点彻底解耦。◆ 补充：主链路响应时间从 X ms 降到 Y ms。

要素二：批量处理与部分失败

Lambda 处理 SQS/Stream 批次时，单条记录失败不应导致整批重试。我们使用 `aws_lambda_powertools` 的 batch 处理工具，支持**部分批次失败**语义，仅将失败记录投入死信队列，正常记录正常提交。

要素三：配置与密钥管理

所有配置（数据库连接、SNS Topic ARN、外部端点）统一从 SSM Parameter Store 读取（`aws/ssm_client.SsmClient`），代码仓库中不落任何明文凭证。服务端侧同样采用 STS AssumeRole 获取临时凭证 + `DefaultCredentialsProvider` 兜底，避免长期凭证泄露风险。

要素四：独立交付流水线

每个 Lambda 拥有独立的 Dockerfile 与独立的 GitHub Actions 工作流（按 `import-*` / `send-*` / `update-travel-status` 命名），支持单函数灰度与回滚，部署动作通过 SAM 完成。这带来一个重要收益：**单个函数的变更不需要整体回归**。

第 4 段 • 难点与解决 (约 700 字)

云原生主题最佳组合：

优先级	难点	适配理由
首选	⑥ Serverless 数据同步链路	主题核心
次选	⑦ 批量推送的幂等与重试	体现 Serverless 下的可靠性设计
三选	⑨ 分布式会话 (Redis Cluster)	体现「无状态化」这一云原生核心原则
加分	⑧ 跨国多时区一致性	跨国部署的特色问题，容易出彩

难点示例 (⑥)：

挑战在于**同步延迟与顺序性**。DynamoDB Stream 保证同一分区的记录有序，但跨分区不保证。若两个业务域的数据存在先后依赖，并行处理可能产生顺序错乱。我们的应对是：把存在依赖关系的实体收敛到同一分区键下（如旅行信息统一以 `travel_no` 为分区键），借由 Stream 的分区内有序性保证处理顺序；对确实跨域的场景，采用**幂等 + 版本号**的设计，使乱序到达的更新可以通过比较版本号判定是否过期，避免旧数据覆盖新数据。

第 5 段 • 总结与不足 (约 200 字)

目前 14 个函数已稳定支撑数据同步，主链路响应时间显著下降，且各函数按需计费，平峰期成本大幅降低。

不足之处有两点：

一是冷启动在低频函数上仍有可感知延迟，尚未引入预置并发；

二是函数间缺少统一的编排视图，跨域数据流向的全局监控还不够直观。

后续计划通过预置并发与链路追踪可视化进一步完善。

通过本项目，我认识到 **Serverless** 并非“银弹”——它适合事件驱动、无状态、突发型的负载，而强一致的事务场景仍需依托关系型数据库。

四、本项目填充点速查

论文要素	直接可写的内容
云平台	AWS ap-northeast-1 (东京)
容器化	各模块独立 Dockerfile，统一 Java 21 基础镜像；Service 暴露 6565
本地云环境	LocalStack (<code>:4566</code>) 模拟 DynamoDB/S3；TestContainers 模拟 MySQL
Serverless 规模	14 个 Lambda 函数，Python 3.13
运行时框架	<code>aws_lambda_powertools</code> (Logger / Tracer / batch 处理)
事件源	DynamoDB Stream (5 个函数) + SQS (6 个函数) + 定时 (3 个函数)
配置管理	SSM Parameter Store (<code>SsmClient</code>)
凭证管理	STS AssumeRole 临时凭证 + DefaultCredentialsProvider
交付方式	每函数独立 Dockerfile + 独立 GitHub Actions + SAM 部署
批处理容器	Batch 模块 <code>web-application-type: none</code> ，无端口占用
监控	Prometheus PushGateway (Batch) ； Powertools Tracer (FaaS)
存储服务	S3 (ap-northeast-1)、DynamoDB (5 表)、Aurora MySQL
消息服务	SNS (<code>app.services.aws.sns</code>)、SQS
单元测试	pytest + moto (模拟 AWS 服务)，覆盖率纳入 CI

五、可用难点素材（按适配度排序）

1. ⑥ **Serverless 数据同步与最终一致性** ★★★ —— 本主题首选
2. ⑦ **批量推送的幂等与重试** ★★★ —— 可靠性设计
3. ⑨ **分布式会话 Redis Cluster** ★★ —— 无状态化
4. ⑧ **跨国多时区一致性** ★★★ —— 跨国部署特色，**强烈建议写**
5. ③ **AWS SDK v1/v2 并存** ★★ —— 云 SDK 演进

六、踩坑提醒

坑	后果	正确做法
把 Serverless 说成万能	缺乏判断力	明确写出 不适用场景 （强一致事务、长时任务、高频稳定负载）
不提冷启动	明显漏考点	主动提，并说明你的缓解手段（本文用「低频函数可接受延迟」+ 预置并发作为改进方向）
编造 K8s 使用经历	高风险	本项目 未见 K8s 编排证据 ，不要写「我们用 K8s 做服务编排」；写 Docker 镜像 + 独立部署即可
编造服务网格 (Istio)	高风险	同上，本项目无 Service Mesh
只讲成本不讲复杂度	片面	写出 Serverless 的代价：本地调试困难、分布式追踪复杂、函数间依赖不直观
忽略 IaC	漏考点	本项目用 SAM 部署、GitHub Actions 流水线，可提「基础设施即代码」
数字全靠编	阅卷老师能看出来	只填你真实知道的；不确定的量级宁可写相对值（如「下降约 40%」）

⚠ 技术真实性红线：BASE HOME 使用 Docker + Lambda + SAM + GitHub Actions，**但没有 Kubernetes、没有 Istio、没有服务网格**。若题目明确问 K8s，请评估后改用其他题目，或如实写「容器编排需求较轻，以单容器部署为主，未引入 K8s」——这本身就是一个合理的架构决策。

配套：01-微服务架构设计.md · 03-数据架构与混合存储.md（同步链路的另一端）